

Блочная модель элемента, плавающие элементы и обтекание

Блочная модель

Если элемент определён как блочный, то он будет вести себя следующим образом:

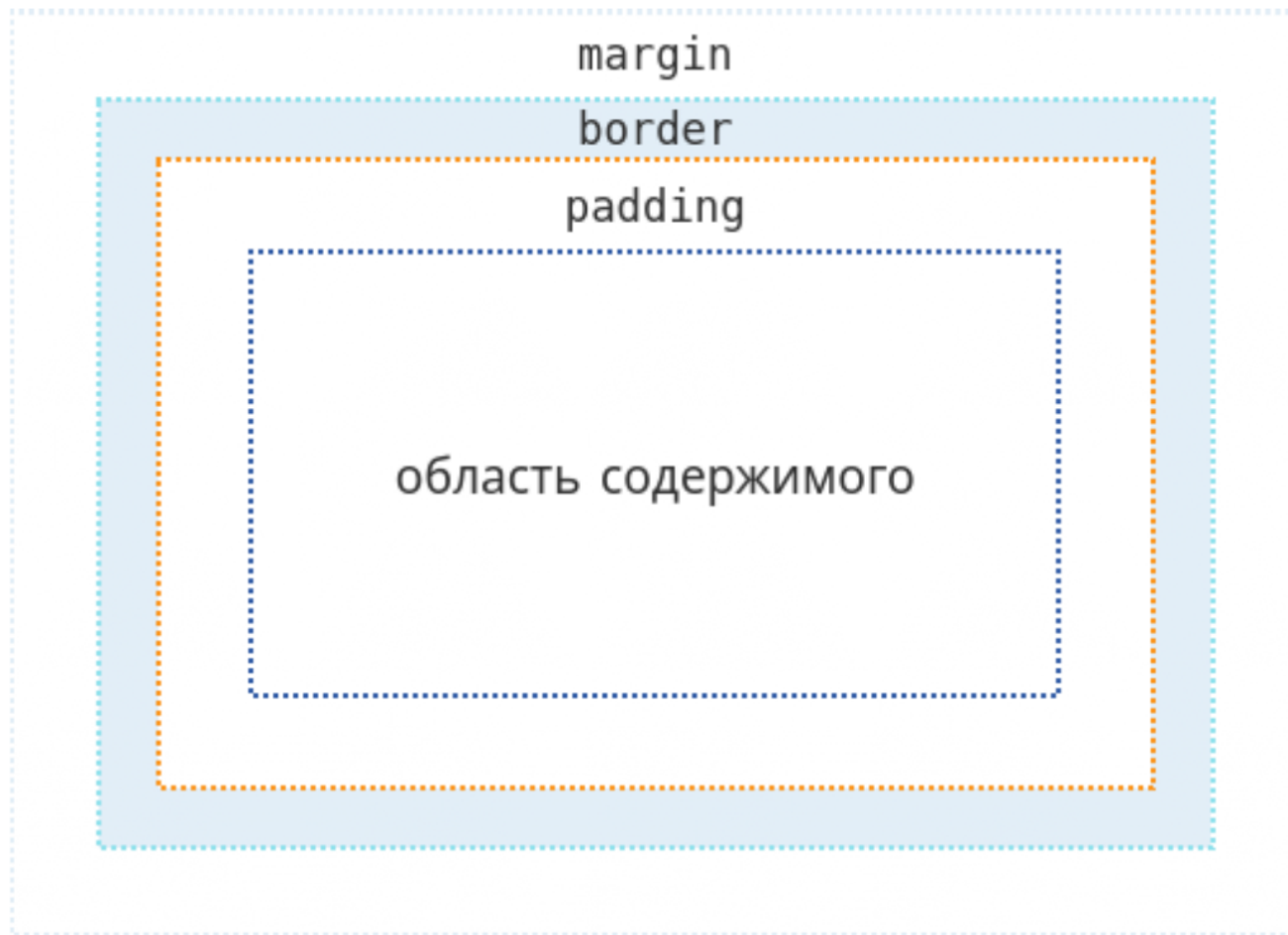
- Начнётся с новой строки;
- Будет расширяться вдоль строки таким образом, чтобы заполнить всё пространство, доступное в её контейнере. В большинстве случаев это означает, что блок станет такой же ширины, как и его контейнер, заполняя 100% доступного пространства;
- Будут применяться свойства **width** и **height**;
- Внешние и внутренние отступы, рамка будут отодвигать от него другие элементы.

Состав блочной модели: content, padding, border, margin

Составляя блочный элемент в CSS мы имеем:

- **Содержимое**: область, где отображается ваш контент, размер которой можно изменить с помощью таких свойств, как **width** и **height**.
- **Внутренний отступ**: отступы располагаются вокруг содержимого в виде пустого пространства; их размер контролируется с помощью **padding** и связанных свойств.
- **Рамка**: рамка оборачивает содержимое и внутренние отступы. Её размер и стиль можно контролировать с помощью **border** и связанных свойств.
- **Внешний отступ**: внешний слой, заключающий в себе содержимое, внутренний отступ и рамки, представляет собой пространство между текущим и другими элементами. Его размер контролируется с помощью **margin** и связанных свойств.

Состав блочной модели: контент, padding, border, margin



Свойства: width, height, box-sizing

Свойство width

- Свойство **width** определяет **ширину содержимого блока**.
- Это свойство не применяется к незамещаемым строчным элементам **display: inline;**
- Ширина содержимого встроенных блоков определяется шириной отображаемого содержимого внутри них. Встроенные блоки сливаются в линейные блоки. Ширина линейных блоков определяется шириной содержащего блока, но может быть уменьшена из-за наличия свойства float.
- **Отрицательные значения не допускаются.**
- Свойство не наследуется.

Свойство width. Синтаксис.

width: 100px;

width: 10em;

width: 50%;

width: auto;

width: inherit;

Свойство width

длина	Ширина элемента задается в единицах длины, например, px, em и т.д.
%	Вычисляется относительно ширины содержащего блока. Для абсолютно позиционированных элементов процент вычисляется с учетом ширины области отступов padding содержащего блока.
auto	Ширина вычисляется в зависимости от значений других свойств. Значение по умолчанию.

Свойства min-width и max-width

- Свойства **min-width** и **max-width** позволяют ограничивать ширину содержимого до определенного диапазона. Значения не могут быть отрицательными. Для **min-width** значение по умолчанию **0**, для **max-width** — **none**.
- Свойства не наследуются.

длина	Задаёт фиксированную минимальную или максимальную используемую ширину.
%	Указывает процент для определения используемого значения. Процент рассчитывается относительно ширины содержащего блока.
none	Означает отсутствие ограничений ширины блока.

Свойства min-width и max-width.

Синтаксис.

min-width: 100px;

min-width: 10em;

min-width: 50%;

min-width: inherit;

max-width: 500px;

max-width: 20em;

max-width: 80%;

max-width: none;

max-width: inherit;

Свойство height

- Свойство **height** определяет высоту содержимого блока. Это свойство не применяется к незамещаемым строчным элементам. Значения длины не могут быть отрицательными.
- Свойство не наследуется.

Длина	Высота области содержимого задается в единицах длины.
%	Задаёт высоту в процентах. Процент рассчитывается относительно высоты содержащего блока. Если высота содержащего блока не указана явно (то есть зависит от высоты содержимого) и этот элемент не является абсолютно позиционированным, значение вычисляется как auto. Для абсолютно позиционированных элементов процент вычисляется с учётом высоты области отступов padding содержащего блока.
auto	Высота зависит от значений других свойств. Значение по умолчанию.

Свойство height. Синтаксис.

```
height: 100px;  
height: 10em;  
height: 50%;  
height: auto;  
width: inherit;
```


Свойства min-height и max-height

- Иногда полезно ограничить высоту элементов определенным диапазоном. Свойства **min-height** и **max-height** предлагают эту функциональность.
- Свойства не наследуются.

длина	Задаёт фиксированную минимальную или максимальную вычисленную высоту в единицах длины. Значения не могут быть отрицательными.
%	Указывает процент для определения используемого значения. Процент рассчитывается относительно высоты содержащего блока. Если высота содержащего блока не указана явно (т.е. зависит от высоты содержимого) и этот элемент не является абсолютно позиционированным, процентное значение обрабатывается как 0 для min-height или none для max-height .
none	Отсутствие ограничений высоты блока, только для max-height .

Свойства min-height и max-height. Синтаксис.

```
min-height: 100px;  
min-height: 2em;  
min-height: 50%;  
min-height: inherit;
```

```
max-height: 500px;  
max-height: 20em;  
max-height: 80%;  
max-height: none;  
max-height: inherit;
```

Свойство box-sizing

- Свойство **box-sizing** переключает блочную модель с фиксированных размеров длины и ширины на **content-box** и **border-box**. Это влияет на интерпретацию всех свойств, определяющих размеры, включая **flex-basis**.
- Свойство не наследуется.

Некоторые HTML-элементы, например, `<button>`, по умолчанию имеют **box-sizing: border-box**

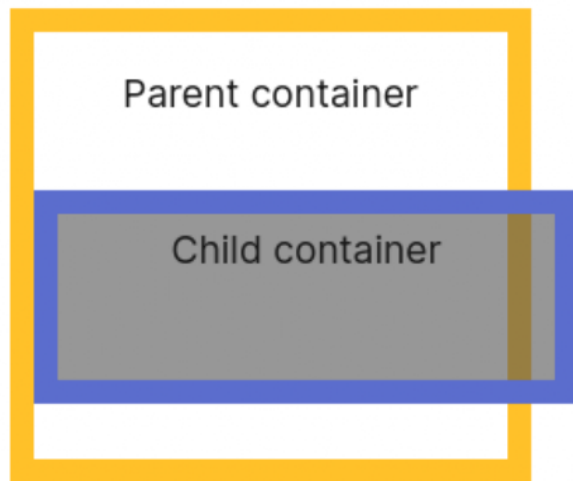
Свойство box-sizing

content-box	Это поведение ширины и высоты, как указано в CSS2.1. Заданные ширина и высота (и соответствующие min/max-свойства) применяются к ширине и высоте области содержимого элемента. Поля padding и рамка border элемента располагаются за пределами указанной ширины и высоты. Значение по умолчанию.
border-box	Любые padding или border, заданные для элемента, размечаются и отрисовываются внутри указанных значений ширины и высоты. Ширина и высота содержимого вычисляются путем вычитания ширины границ и полей соответствующих сторон из указанных свойств ширины и высоты. Значение auto свойств width и height не зависит от свойства box-sizing и всегда устанавливает размер блока с содержимым. Сумма padding и border не должна превышать заданные значения width и height, в противном случае размер области содержимого будет равен нулю.
initial	Устанавливает значение свойства в значение по умолчанию.

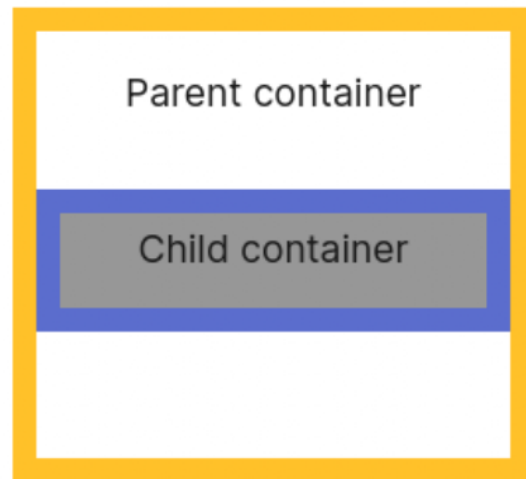
Свойство box-sizing. Синтаксис.

```
box-sizing: content-box;  
box-sizing: border-box;  
box-sizing: inherit;  
box-sizing: initial;
```

```
box-sizing: content-box;  
width: 100%;  
border: solid #5b6dcd 10px;  
padding: 5px;
```



```
box-sizing: border-box;  
width: 100%;  
border: solid #5b6dcd 10px;  
padding: 5px;
```



Свойство Content

Свойство Content

- CSS **content** генерирует содержимое, которое визуально отображается на экране монитора, не добавляясь к дереву документа DOM. Программы для чтения с экрана не имеют доступ к содержимому, созданному с использованием псевдоэлементов и не могут его прочесть, поэтому рекомендуется не использовать псевдоэлементы для вставки важного контента на страницу.
- Содержимое, вставляемое с помощью свойства **content**, появляется внутри элемента, до или после его содержимого. С помощью CSS можно генерировать содержимое следующими способами:
 - с помощью свойства **content** в сочетании с псевдоэлементами **::before** и **::after**;
 - с помощью свойств **counter-increment** и **counter-reset**.

Свойство Content

- В основе генерируемого содержимого лежат псевдоэлементы **::before** или **::after**.
- Псевдоэлементы создают **абстракции** о дереве документа помимо тех, которые определены языком документа, в данном случае — **HTML**.
- Например, HTML **не предлагает механизмы** доступа к первой букве или первой строке содержимого элемента. Псевдоэлементы **CSS позволяют ссылаться** на эту не имеющую доступа информацию.
- Псевдоэлементы также предоставляют дизайнерам стилей способ **присвоить стиль содержимому, которого нет в исходном документе**.

```
h1::before, h1::after {content: "";} 
```


Свойство Content

normal	Значение по умолчанию, означает отсутствие добавляемого содержимого.
none	Не добавляет содержимое. Используется в случае, когда нужно удалить генерируемое содержимое для одного элемента из группы элементов (например, элементы списка), для которых уже задано это свойство.
counter()	Даёт возможность создавать счётчики, задавая для них точку отсчёта и приращение на некоторую величину с помощью свойства counter-reset. Для прямого увеличения счёта необходимо использовать свойство counter-increment.
attr()	Добавляет до или после элемента значение атрибута, заключённого в скобки. Чтобы вставить пробел между основным содержимым и генерируемым, нужно добавить пробел перед скобкой или после нее, например, content: attr(href);

Свойство Content

" "	Текст, который добавляется на веб-страницу, должен быть заключен в двойные или одинарные кавычки. Пустые кавычки можно использовать для добавления блочного содержимого.
open-quote	Добавляет к содержимому открывающую кавычку.
close-quote	Добавляет к содержимому закрывающую кавычку.
no-open-quote	Удаляет открывающую кавычку, при этом уровень их вложенности продолжает учитываться.
no-close-quote	Удаляет закрывающую кавычку.
url()	Добавляет медиа-содержимое, например, изображение, звук, видео. В качестве значения атрибута в скобках указывается адрес внешнего ресурса, который вставляется в выбранное место документа.

СВОЙСТВО Content. Синтаксис.

```
h1::before, h1::after {  
  content: "Yay!";  
  font-family: 'Dancing Script', cursive;  
  color: #f7b21c;  
  text-shadow: 1px 1px 2px grey;  
}  
h1::before {  
  margin-right: 30px;  
}  
h1::after {  
  margin-left: 30px;  
}
```



Yay! **Vacation soon!** *Yay!*

Отступы элемента

Отступы элемента

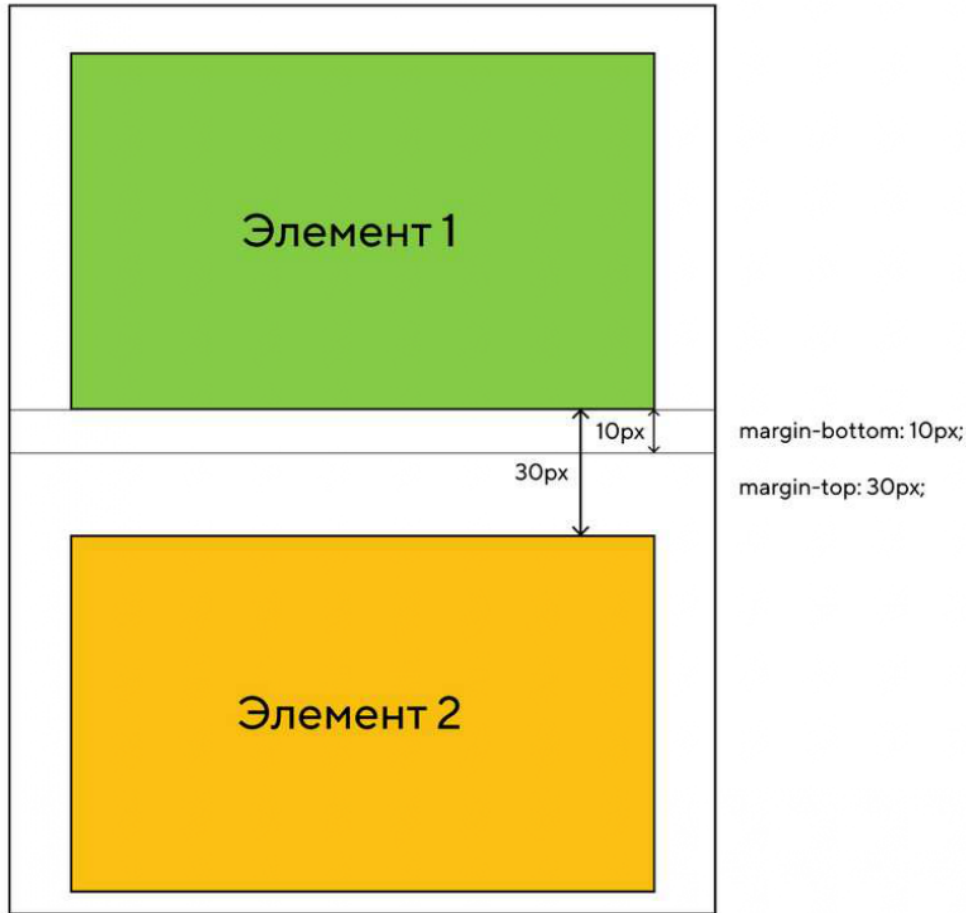
- Отступы **огибают край рамки** элемента, обеспечивая расстояние между соседними блоками. Свойства отступов определяют их толщину.
- Применяются ко всем элементам, кроме внутренних элементов таблицы.
- Сокращенное свойство **margin** **задает отступы для всех четырех сторон**, а его подсвойства задают отступ только для соответствующей стороны.
- **Смежные вертикальные отступы** элементов в блочной модели **схлопываются**.

Схлопывание вертикальных отступов

- **Смежные вертикальные отступы** двух или более элементов уровня блока margin **объединяются** (перекрываются). При этом ширина общего отступа равна ширине большего из исходных. Исключение составляют отступы корневого элемента, которые не схлопываются.
- **Объединение отступов выполняется только для блочных элементов в нормальном потоке документа**. Если среди схлопывающихся отступов есть отрицательные значения, то браузер добавит отрицательное значение к положительному, а полученный результат и будет расстоянием между элементами. Если положительных отступов нет, то максимум абсолютных значений соседних отступов вычитается из нуля.

Схлопывание вертикальных отступов.

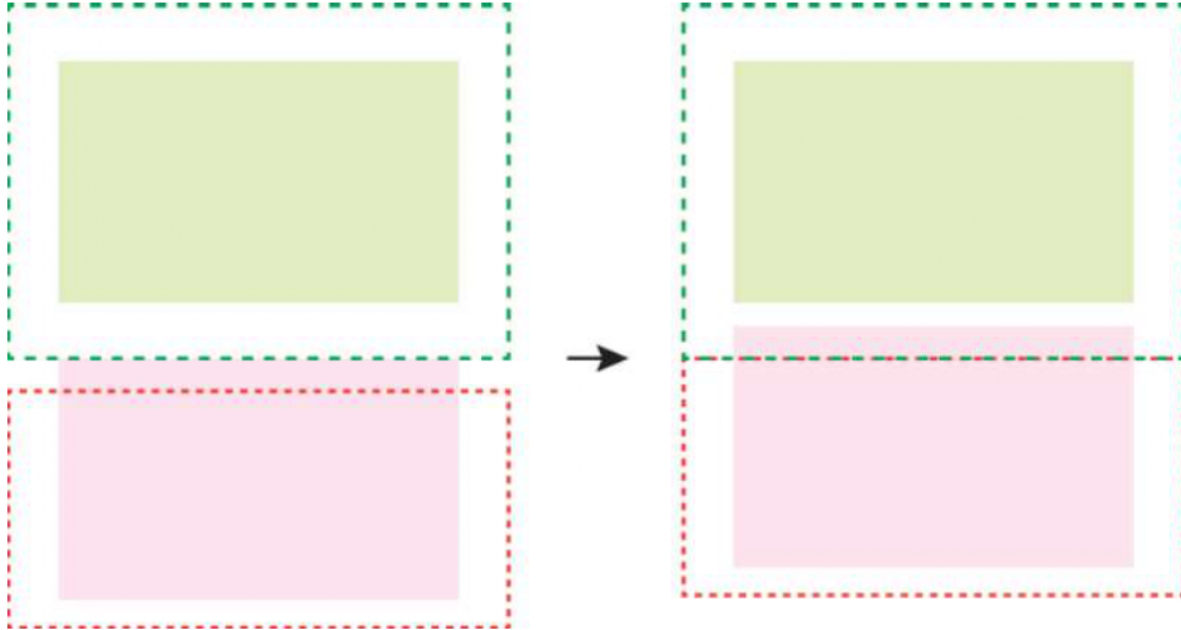
Примеры



- Нижний отступ первого элемента **схлопывается** с верхним отступом следующего элемента;
 - Если оба значения **margin** положительные, то из них выбирается наибольшее значение и оно задаётся как расстояние между блоками.
- ```
.block1 {
 background: #a8d51f; /* Цвет фона первого блока */
 margin-bottom: 10px; /* Отступ снизу */
}
.block2 {
 background: #f4c51a; /* Цвет фона второго блока */
 margin-top: 30px; /* Отступ сверху */
}
```

# Схлопывание вертикальных отступов.

## Примеры



- верхний **margin** у нижнего блока отрицательный.  
Если один из **margin** отрицательный, то в этом случае происходит их складывание по правилам математики:  $x + (-y) = x - y$
- Если полученное значение в результате суммирования окажется **отрицательным**, то оно будет действовать на нижний блок, соответственно, он сдвинется вверх на указанное значение. При этом возможно наложение одного блока на другой.

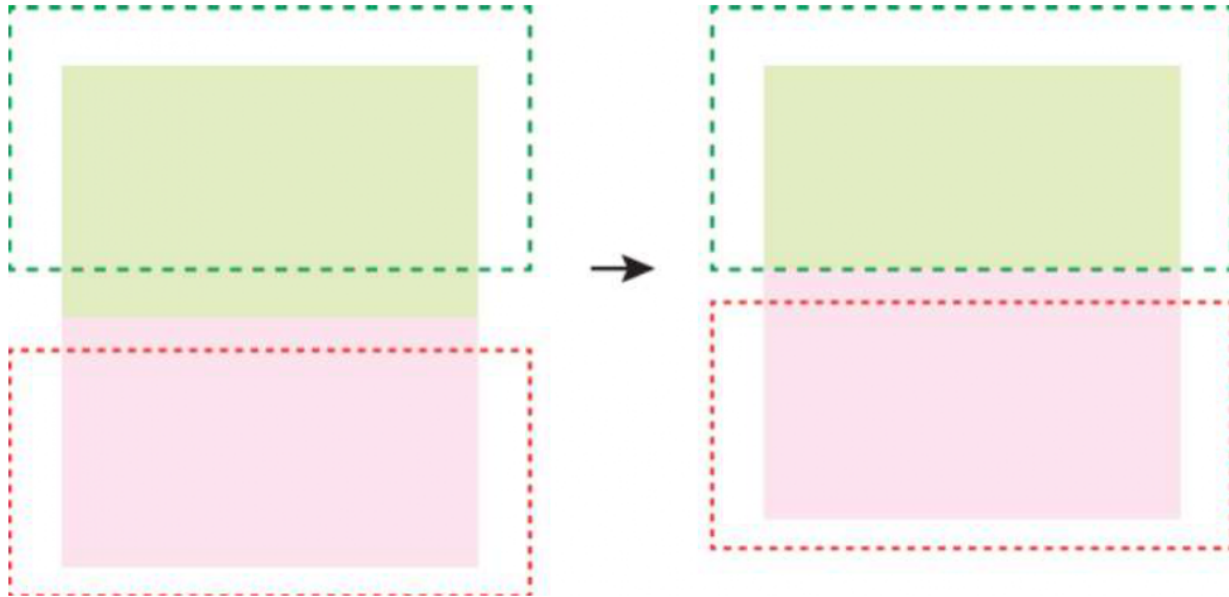
```
.block1 {
 background: #E2EDC1; /* Цвет фона первого блока */
 margin-bottom: 20px; /* Отступ снизу */
}
```

```
.block2 {
 background: #FCE3EE; /* Цвет фона второго блока */
 margin-top: -10px; /* Отрицательный отступ сверху */
}
```



# Схлопывание вертикальных отступов.

## Примеры



- Если оба **margin** отрицательные, то из двух значений выбирается наибольшее по модулю, оно же и выступает в качестве отрицательного отступа между элементами.
- Так, если отступы равны **-10px** и **-20px**, то итоговое значение будет **-20px**. При этом элементы будут частично перекрываться

```
.block1 {
 background: #E2EDC1; /* Цвет фона первого блока */
 margin-bottom: -20px; /* Отступ снизу */ }
.block2 {
 background: #FCE3EE; /* Цвет фона второго блока */
 margin-top: -10px; /* Отрицательный отступ сверху */
}
```

# Схлопывание вертикальных отступов

Отступы **не схлопываются**:

- Между плавающим блоком и любым другим блоком;
- У плавающих элементов и элементов со значением **overflow**, отличным от **visible**, со своими дочерними элементами в потоке;
- У абсолютно позиционированных элементов, даже с их дочерними элементами;
- У строчно-блочных элементов.

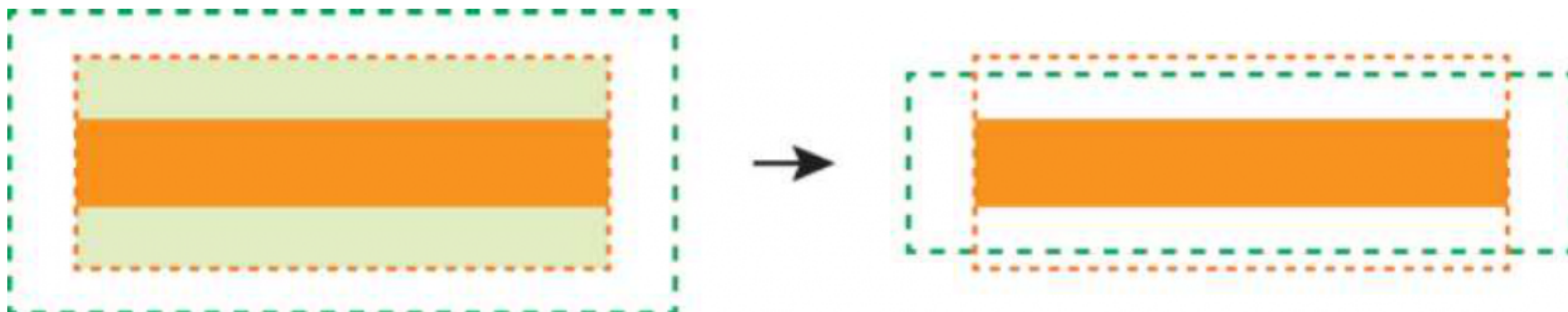
**\* Для предотвращения проблемы схлопывания рекомендуется задавать для всех элементов только верхний или нижний margin.**

# Выпадение отступов. Примеры

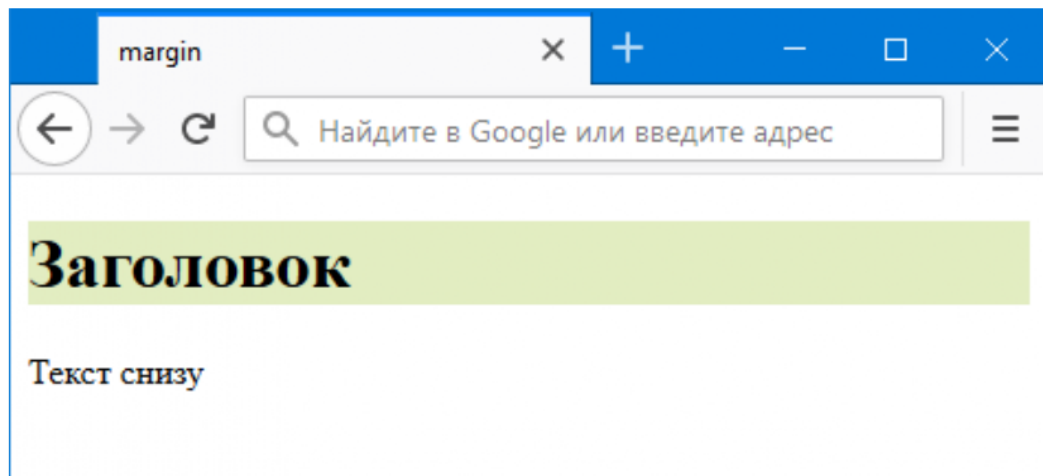
Бывают случаи, когда отступы не схлопываются, а **выпадают** из контейнера. Более точно комбинируется так:

- **margin-top** первого дочернего элемента выпадает из **margin-top** родителя;
- **margin-bottom** последнего дочернего элемента из **margin-bottom** родителя.

Такое поведение часто можно встретить у заголовков, вроде <h1> или <h2>, которые идут первыми в блоке. У этих заголовков уже содержится **margin-top** и **margin-bottom** по умолчанию и он, объединяясь с родительским **margin**, влияет на отступы всего блока.



# Выпадение отступов. Примеры



```
<div class="block">
 <h1>Заголовок</h1>
</div>
<p>Текст снизу</p>
```

```
.block{
 background: #FCE3EE; /* Цвет фона блока */
}
h1{
 background: #E2EDC1; /* Цвет фона заголовка
*/
}
```

# Выпадение отступов.

Выпадение не действует на дочерние элементы:

- если у родителя значение **overflow** задано как **auto**, **hidden** или **scroll**;
- если у родителя на стороне схлопывания задано свойство **padding**;
- если у родителя на стороне схлопывания задано свойство **border**.

# Свойства margin-top, margin-right, margin-bottom, margin-left

- Свойства устанавливают верхний, правый, нижний и левый отступ блока элемента **соответственно**. Отрицательные значения допускаются, но могут существовать ограничения для конкретной реализации.
- Свойства не наследуются.

# Свойства `margin-top`, `margin-right`, `margin-bottom`, `margin-left`

длина	Размер отступа задается в единицах длины, например, px, in, em. Значение по умолчанию 0.
%	Вычисляется относительно ширины блока контейнера. Изменяются, если изменяется ширина родительского элемента.
auto	Для элементов уровня строки, плавающих ( <code>float</code> ) значения <code>margin-left</code> или <code>margin-right</code> вычисляются в 0. Если для элементов уровня блока задано <code>margin-left: auto</code> или <code>margin-right: auto</code> — соответствующее поле расширяется до края содержащего блока, если оба — их значения становятся равными, что горизонтально центрирует элемент относительно краев содержащего блока.
initial	Устанавливает значение свойства в значение по умолчанию.

# Свойства margin-top, margin-right, margin-bottom, margin-left. Синтаксис.

```
margin-top: 20px;
margin-right: 1em;
margin-bottom: 5%;
margin-left: auto;
margin-top: inherit;
margin-right: initial;
```



# Краткая запись отступов: свойство `margin`

- Свойство `margin` является сокращенным свойством для установки `margin-top`, `margin-right`, `margin-bottom` и `margin-left` в одном объявлении.
- Если существует только одно значение, оно применяется ко всем сторонам.
- Если два — верхний и нижний отступы устанавливаются на первое значение, а правый и левый — устанавливаются на второе.
- Если имеется три значения — верхний отступ устанавливается на первое значение, левый и правый — на второе, а нижний — на третье.
- Если есть четыре значения — они применяются сверху, справа, снизу и слева соответственно.

# Краткая запись отступов: свойство `margin`

- Свойство `margin` является сокращенным свойством для установки `margin-top`, `margin-right`, `margin-bottom` и `margin-left` в одном объявлении.

<code>margin: 10px</code>	Если существует только <b>одно</b> значение, оно применяется ко всем сторонам.
<code>margin: 10px 10px</code>	Если <b>два</b> — верхний и нижний отступы устанавливаются на первое значение, а правый и левый — устанавливаются на второе.
<code>margin: 10px 10px 10px</code>	Если имеется <b>три</b> значения — верхний отступ устанавливается на первое значение, левый и правый — на второе, а нижний — на третье.
<code>margin: 10px 10px 10px 10px</code>	Если есть <b>четыре</b> значения — они применяются сверху, справа, снизу и слева соответственно.

Поля элемента

# Поля элемента

- Область полей представляет собой пространство между краем области содержимого и рамкой элемента. Свойства полей определяют толщину их области. Применяются ко всем элементам, кроме внутренних элементов таблицы (за исключением ячеек таблицы). Сокращенное свойство **padding** задает поля для всех четырех сторон, а **подсвойства** устанавливают только их соответствующие стороны.
- Фоны элемента по умолчанию закрашивают поля элемента и пространство под его рамкой. Это поведение можно настроить с помощью свойств **background-origin** и **background-clip**.

# Свойства padding-top, padding-right, padding-bottom, padding-left

- Свойства устанавливают верхнее, правое, нижнее и левое поля соответственно. Отрицательные значения недопустимы.
- Свойства не наследуются.

длина	Поля элемента задаются при помощи единиц длины, например, px, pt, cm. Значение по умолчанию 0.
%	Вычисляются относительно ширины родительского элемента, могут меняться при изменении ширины элемента. Поля сверху и снизу равны полям слева и справа, т.е. верхние и нижние поля тоже вычисляются относительно ширины элемента.
initial	Устанавливает значение свойства в значение по умолчанию.

# Свойства padding-top, padding-right, padding-bottom, padding-left. Синтаксис.

```
padding-top: 0.5em;
padding-right: 0;
padding-bottom: 2cm;
padding-left: 10%;
padding-top: inherit;
padding-bottom: initial;
```

# Краткая запись полей: свойство padding

- Свойство **padding** является сокращенным свойством для установки **padding-top**, **padding-right**, **padding-bottom** и **padding-left** в одном объявлении.

<code>padding: 10px</code>	Если существует только <b>одно</b> значение, оно применяется ко всем сторонам.
<code>padding: 10px 10px</code>	Если есть <b>два значения</b> , верхнее и нижнее поля устанавливаются на первое значение, а правое и левое — на второе.
<code>padding: 10px 10px 10px</code>	Если имеется <b>три значения</b> , верхнее поле устанавливается на первое значение, левое и правое — на второе, а нижнее — на третье.
<code>padding: 10px 10px 10px 10px</code>	Если есть <b>четыре значения</b> — они применяются сверху, справа, снизу и слева соответственно.

Рамки: CSS-рамка



# Рамки: CSS-рамка

- **CSS-рамка** элемента представляет собой одну или несколько линий, окружающих содержимое элемента и его поля **padding**. Рамка задаётся с помощью краткого свойства **border**. Стиль рамки задается с помощью трех свойств: стиль, цвет и ширина.

# Стиль рамки border-style

- По умолчанию рамки всегда **отрисовываются поверх фона элемента**, фон распространяется до внешнего края элемента.
- Стиль рамки определяет ее отображение, **без этого свойства** рамки **не будут видны вообще**.
- Для элемента можно задавать рамку для всех сторон одновременно с помощью свойства **border-style** или для каждой стороны отдельно с помощью уточняющих свойств **border-top-style** и т.д.
- Не наследуется.

# Стиль рамки border-style

none

Значение по умолчанию, означает отсутствие рамки. Также убирает рамку элемента из группы элементов с установленным значением данного свойства.

hidden

Эквивалентно `none`.

dotted

dotted

dashed

dashed

solid

solid

double

double

groove

groove

# Стиль рамки border-style

ridge

ridge



inset

inset



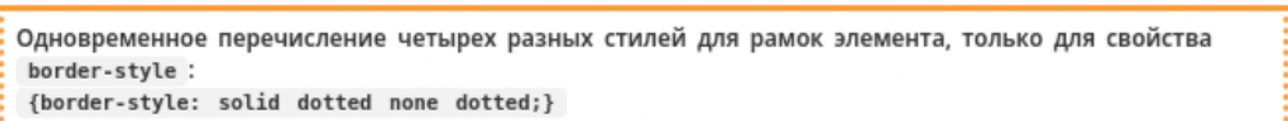
outset

outset



{1,4}

Одновременное перечисление четырех разных стилей для рамок элемента, только для свойства  
`border-style :`  
`{border-style: solid dotted none dotted;}`



initial

Устанавливает значение свойства в значение по умолчанию.

# Стиль рамки border-style. Синтаксис.

```
p {border-style: solid;}
```

```
p {border-top-style: solid;}
```

# Цвет рамки border-color

- Свойство задаёт цвет рамок всех сторон одновременно. С помощью уточняющих свойств можно установить свой цвет для рамки каждой стороны элемента.
- Если для рамки цвет не задан, то он будет таким же, как и цвет текста элемента.
- Если в элементе нет текста, то цвет рамки будет таким же, как и цвет текста родительского элемента.
- Не наследуется.

# Цвет рамки border-color

transparent

Устанавливает прозрачный цвет для рамки. При этом ширина рамки остается. Можно использовать для смены цвета рамки при наведении курсора мыши на элемент, чтобы избежать смещение элемента.

color

Цвет рамок задается при помощи значений свойства [color](#).

```
{border-color: #cacd58;}
```

{1, 4}

Одновременное перечисление четырех разных цветов для рамок элемента, только для свойства border-color :

```
{border-color: #cacd58 #5faf8a #b9cea5 #aab238;}
```

initial

Устанавливает значение свойства в значение по умолчанию.

# Цвет рамки border-color. Синтаксис.

```
p {border-color: #cacd58;}
```



# Ширина рамки border-width

- Ширина рамки задается с помощью единиц измерения длины или **ключевых слов**. Если для свойства **border-style** задано значение **none**, и для рамки элемента установлена какая-то ширина, то в данном случае ширина рамки приравнивается к нулю.
- Не наследуется.

# Ширина рамки border-width

thin /  
medium /  
thick

Ключевые слова, устанавливают ширину рамки относительно друг друга. Первое значение уже, чем второе, второе — тоньше третьего. Значение по умолчанию — `medium`

width (px,  
em)

```
{border-width: 5px;}
```

{1,4}

Возможность одновременного задания четырех разных ширин для рамок элемента, только для свойства `border-width` :

```
{border-width: 5px 10px 15px 3px;}
```

initial

Устанавливает значение свойства в значение по умолчанию.

# Ширина рамки border-width. Синтаксис.

```
p {border-width: 2px;}
```

# Задание рамки одним свойством

Свойство **border** позволяет объединить в себе следующие свойства: **border-width**, **border-style**, **border-color**, например:

```
div {
 width: 100px;
 height: 100px;
 border: 2px solid grey;
}
```

При этом заданные свойства будут применяться ко всем границам элемента одновременно. Если какое-то из значений не указано, его место займет значение по умолчанию.

# Задание рамки для одной границы элемента

- В случае, когда необходимо задать разный стиль границ элемента, можно воспользоваться краткой записью для соответствующей границы.
- Перечисленные ниже свойства объединяют в одно объявление следующие свойства: **border-width**, **border-style** и **border-color**. Перечень свойств указывается в заданном порядке, при этом одно или два значения могут быть пропущены, в этом случае их значения примут значения по умолчанию.
- Стиль верхней границы задается с помощью свойства **border-top**, нижней — **border-bottom**, левой — **border-left**, правой — **border-right**.

# Задание рамки для одной границы элемента. Синтаксис.

```
p {border-top: 2px solid grey;}
```

# Закругление углов с помощью border-radius

- **Свойство позволяет закруглить углы строчных и блочных элементов.**  
Кривая для каждого угла определяется с помощью одного или двух радиусов, определяющих его форму — круга или эллипса. Радиус распространяется на весь фон, даже если элемент не имеет границ, точное положение секущей определяется с помощью свойства **background-clip**.
- Свойство **border-radius** позволяет закруглить все углы одновременно, а с помощью свойств **border-top-left-radius**, **border-top-right-radius**, **border-bottom-right-radius**, **border-bottom-left-radius** можно закруглить каждый угол отдельно.
- Если задать **два значения** для свойства **border-radius**, то первое значение закруглит верхний левый и нижний правый угол, а второе — верхний правый и нижний левый.
- **Значения, заданные через /**, определяют горизонтальные и вертикальные радиусы.
- Свойство не наследуется

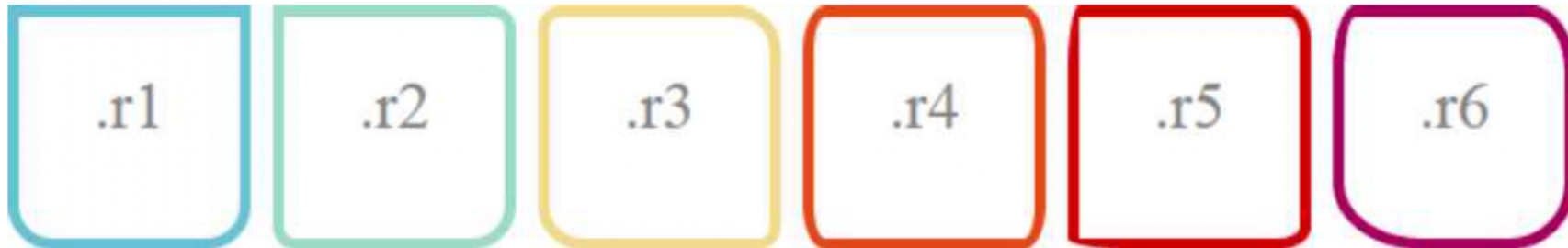
# Закругление углов с помощью border-radius

длина	Позволяет закруглить углы блока с помощью значений единиц длины — px, em.
%	Значения, закругляющие углы, задаются в процентах от длины и ширины сторон элемента.
initial	Устанавливает значение свойства в значение по умолчанию.



# Закругление углов с помощью border-radius. Синтаксис.

```
div {width: 100px; height: 100px; border: 5px solid;}
.r1 {border-radius: 0 0 20px 20px;}
.r2 {border-radius: 0 10px 20px;}
.r3 {border-radius: 10px 20px;}
.r4 {border-radius: 10px/20px;}
.r5 {border-radius: 5px 10px 15px 30px/30px 15px 10px 5px;}
.r6 {border-radius: 10px 20px 30px 40px/30px;}}
```



# Закругление углов с помощью border-radius. Синтаксис.

```
div {width: 100px; height: 100px; border: 5px solid;}
.r7 {border-radius: 50%;}
.r8 {border-top: none; border-bottom: none; border-radius: 30px/90px;}
.r9 {border-bottom-left-radius: 100px;}
.r10 {border-radius: 0 100%;}
.r11 {border-radius: 0 50% 50% 50%;}
.r12 {border-top-left-radius: 100% 20px; border-bottom-right-radius: 100% 20px;}
```



Видимость: display

# Свойство display

- Свойство **display** отвечает за вывод и визуальное отображение элементов на странице. Также с помощью свойства **display** можно изменить тип генерируемого контейнера. Свойство не наследуется.
- Любой **html-элемент** генерирует на веб-странице прямоугольный контейнер. Все видимое на экране состоит из контейнеров разных типов.
- В **нормальном потоке** блочные элементы генерируют структурные блоки и выводятся вертикально один над другим, занимая по ширине 100% ширины блока-контейнера.

# Свойство display

- Строковые контейнеры генерируют строковые блоки и выводятся в строке **горизонтально**. Ширина строковых элементов равна ширине их содержимого.
- Строчно-блочный элемент также генерирует строку текста, при этом низ элемента **располагается на базовой линии строки текста и не разрывает строку**. Содержимое элемента форматируется так же, как и для блочных элементов, а ширина блока равна ширине содержимого.
- **Таблицы обрабатываются браузером как блоки**. Внутренние элементы таблицы генерируют прямоугольные блоки, имеющие содержимое, отступы padding и рамки border, но не имеющие полей margin.

# Свойство display

<b>inline</b>	Значение по умолчанию. Элемент генерирует строковый блок. Аналог — тег <code>&lt;span&gt;</code> .
<b>block</b>	Элемент генерирует структурный блок, как и тег <code>&lt;div&gt;</code> .
<b>flex</b>	Элемент генерирует структурный блок, который создает адаптивный контейнер для дочерних элементов.
<b>inline-block</b>	Элемент генерирует строковый блок.
<b>inline-flex</b>	Элемент генерирует строковый блок, который создает адаптивный контейнер для дочерних элементов.
<b>inline-table</b>	Элемент определяет структурный блок, который генерирует строковый блок.
<b>list-item</b>	Элемент генерирует структурный блок, который отображается как элемент списка <code>&lt;li&gt;</code> .
<b>table</b>	Элемент генерирует структурный блок. На странице ведет себя аналогично <code>&lt;table&gt;</code> .

# Свойство display

<b>table-caption</b>	Элемент генерирует основной заголовок таблицы. На странице ведет себя аналогично <caption>.
<b>table-column</b>	Элемент описывает столбец ячеек, визуальное представление не генерируется. Аналог — <col>.
<b>table-column-group</b>	Элемент объединяет один или несколько столбцов. Аналог — <colgroup>.
<b>table-cell</b>	Элемент генерирует отдельную ячейку таблицы, на странице ведет себя аналогично <th> и <td>.
<b>table-header-group</b>	Элемент определяет группу строк заголовка, которая всегда отображается перед остальными строками и группами строк. Аналог — <thead>.
<b>table-footer-group</b>	Элемент определяет группу строк заголовка, которая всегда отображается после всех остальных строк и перед любым нижним основным заголовком. Ведет себя аналогично <tfoot>.

# Свойство display

<b>table-row-group</b>	Элемент объединяет одну или несколько строк. Аналог — <tbody>.
<b>table-row</b>	Элемент является строкой ячеек. Пример — <tr>.
<b>none</b>	Элемент не генерирует никакой контейнер, полностью удаляясь со страницы.



# CSS-позиционирование

# CSS-позиционирование

**CSS** рассматривает макет **html-документа** как дерево элементов. Уникальный элемент, у которого нет родительского элемента, называется **корневым элементом**. Модуль CSS-позиционирование описывает, как любой из элементов может быть размещен независимо от порядка документа (т.е. извлечен из «потока»).

В **CSS2** каждый элемент в дереве документа генерирует ноль или более блоков в соответствии с блочной моделью. Модуль **CSS3** дополняет и расширяет схему позиционирования. Расположение этих блоков регулируется:

- размерами и типом элемента,
- схемой позиционирования (нормальный поток, обтекание и абсолютное позиционирование),
- отношениями между элементами в дереве документа,
- внешней информацией (например, размер области просмотра, внутренними размерами изображений и т.д.).

# Схема позиционирования

- **Нормальный поток**

Нормальный поток включает блочный контекст форматирования (элементы с `display block`, `list-item` или `table`), строчный (встроенный) контекст форматирования (элементы с `display inline`, `inline-block` или `inline-table`), и относительное и «липкое» позиционирование элементов уровня блока и строки.

- **Обтекание**

В обтекающей модели блок удаляется из нормального потока и позиционируется влево или вправо. Содержимое обтекает правую сторону элемента с `float: left` и левую сторону элемента с `float: right`.

- **Абсолютное позиционирование**

В модели абсолютного позиционирования блок полностью удаляется из нормального потока и ему присваивается позиция относительно содержащего блока. Абсолютное позиционирование реализуется с помощью значений `position: absolute`; и `position: fixed`;

- Элементом «вне потока» может быть плавающий, абсолютно позиционированный или корневой элемент.

Плавающие элементы и обтекание

# Свойство float

- Обтекание позволяет блокам смещаться влево или вправо на текущей строке. «Плавающий блок» смещается влево или вправо до тех пор, пока его внешний край не коснется края содержащего блока или внешнего края другого плавающего блока. Если имеется линейный блок, внешняя верхняя часть плавающего блока выравнивается с верхней частью текущего линейного блока.
- При использовании свойства **float** для элементов рекомендуется задавать ширину. Тем самым браузер создаст место для другого содержимого. Если для плавающего элемента недостаточно места по горизонтали, он будет смещаться вниз до тех пор, пока не уместится. При этом остальные элементы уровня блока будут его игнорировать, а элементы уровня строки будут смещаться вправо или влево, освобождая для него пространство и обтекая его.
- Правила, регулирующие поведение плавающих боков, описываются свойством **float**.
- Свойство не наследуется.

# Свойство float

<b>none</b>	Отсутствие обтекания. Значение по умолчанию.
<b>left</b>	Элемент перемещается влево, содержимое обтекает плавающий блок по правому краю.
<b>right</b>	Элемент перемещается вправо, содержимое обтекает плавающий блок по левому краю.

# Свойство float

- Плавающий блок принимает размеры своего содержимого с учетом внутренних отступов и рамок. Верхние и нижние отступы margin плавающих элементов не схлопываются.
- Плавающие элементы могут использовать отрицательные отступы margin, чтобы перемещаться за пределы области содержимого их родительского элемента.
- Свойство автоматически изменяет вычисляемое (отображаемое в браузере) значение свойства display на display: block для следующих значений: inline, inline-block, table-row, table-row-group, table-column, table-column-group, table-cell, table-caption, table-header-group, table-footer-group. Значение inline-table меняет на table.
- Свойство не оказывает влияние на элементы с display: flex и display: inline-flex. Не применяется к абсолютно позиционированным элементам.

# Свойство float. Синтаксис.

**БЛОК 1.** Этот блок отображается в нормальном нисходящем потоке.

**БЛОК 2.** Для этого блока задано обтекание по левому краю.

**БЛОК 3.** Этот блок также отображается в нормальном нисходящем потоке. Он игнорирует плавающий блок, а текст в блоке его обтекает.

## РАЗМЕТКА:

```
<div style="height:150px; border:1px dashed #8bc63e">БЛОК 1.</div>
<div style="height:200px; width:300px; border:3px dashed #e03c32; float:left">БЛОК 2.</div>
<div style="height:150px; border:1px solid #888888">БЛОК 3.</div>
```



# Управление потоком рядом с плавающими элементами: свойство `clear`

- Свойство `clear` указывает, какие стороны блока/блоков элемента не должны прилегать к плавающим блокам, находящимся выше в исходном документе. В CSS2 и CSS 2.1 свойство применяется только к неплавающим элементам уровня блока.
- Свойство не наследуется.
- Для предотвращения отображения фона или границ под плавающими элементами используется правило `{overflow: hidden;}`.

# Управление потоком рядом с плавающими элементами: свойство clear

<b>none</b>	Означает отсутствие ограничений на положение элемента относительно плавающих блоков. Значение по умолчанию.
<b>left</b>	Смещает элемент вниз относительно нижнего края любого плавающего слева элемента, находящегося выше в исходном документе.
<b>right</b>	Смещает элемент вниз относительно нижнего края любого плавающего справа элемента, находящегося выше в исходном документе.
<b>both</b>	Смещает элемент вниз относительно нижнего края любого плавающего слева и справа элемента, находящегося выше в исходном документе.

# Свойство clear. Синтаксис.

```
clear: none;
clear: left;
clear: right;
clear: both;
clear: inherit;
```

# Clearfix-хаки

- **Clearfix-хаки** — вспомогательные методы, помогающие нам разрешать ситуации со схлопывающейся высотой у родительского div'a когда внутри него расположенный плавающие элементы например с **float:left**.

## Проблема

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



## Решение

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



# Clearfix-хаки. Метод дополнительного пустого элемента.

- **Суть метода** заключается в том, что в самый конец родительского контейнера, сразу после всех плавающих элементов, вручную добавляется отдельный HTML-тег (обычно пустой блок **div**). Этому элементу через стили задается команда очистки потока.
- **Как это работает:** Браузер видит этот элемент после всех обтекаемых блоков и понимает, что ниже него поток документа должен продолжаться в обычном режиме, поэтому растягивает родителя до этого момента.
- **Недостатки:** Этот подход считается устаревшим и неправильным с точки зрения семантики. Он засоряет HTML-код лишними тегами, которые не несут смысловой нагрузки, что усложняет поддержку проекта и ухудшает доступность сайта для вспомогательных технологий.

# Clearfix-хаки. Метод дополнительного пустого элемента.

```
<div class="container">
 <div class="box" style="float: left;">Блок 1</div>
 <div class="box" style="float: left;">Блок 2</div>
 <!-- Добавляем очищающий элемент -->
 <div class="clear"></div>
</div>
```

```
.clear {
 clear: both; /* Принудительная очистка потока */
}
```

Последний div заставляет контейнер растянуться до своей высоты.

# Clearfix-хаки. Метод свойства переполнения (Overflow).

- **Как это работает:** У родительского блока изменяется свойство, отвечающее за обработку контента, выходящего за границы. При установке значения, отличного от стандартного (например, скрывание переполнения или автоматическое добавление прокрутки), браузер вынужден создать новый контекст форматирования. В рамках этого контекста контейнер начинает автоматически обхватывать свои дочерние элементы с обтеканием.
- **Недостатки:** У метода есть серьезные побочные эффекты. Если выбрать скрывание переполнения, то любые элементы, которые должны выступать за границы контейнера (например, тени, выпадающие списки или абсолютное позиционирование), будут обрезаны. Если выбрать автоматическую прокрутку, может появиться лишняя полоса прокрутки там, где она не нужна.

# Clearfix-хаки. Метод свойства переполнения (Overflow).

```
<div class="container">
 <div class="box" style="float: left;">Блок 1</div>
 <div class="box" style="float: left;">Блок 2</div>
</div>
```

```
.container {
 overflow: hidden; /* Или overflow:
auto */
}
```

Свойство overflow создает новый контекст форматирования, заставляя контейнер обхватить floated-элементы.



# Clearfix-хаки. Метод вспомогательного псевдокласса.

- **Как это работает:** Вместо добавления лишнего тега в HTML, разработчик использует возможности CSS для создания виртуального элемента после содержимого контейнера. Этому псевдоэлементу задается невидимое содержимое и команда очистки потока. Визуально пользователь ничего не замечает, но для браузера это равносильно наличию очищающего блока в конце контейнера.
- **Недостатки:** Единственный минус — необходимость добавлять специальный класс **clearfix** к каждому контейнеру, где используется обтекание. Однако это решение лишено побочных эффектов метода с переполнением и не засоряет HTML-разметку.

# Clearfix-хаки. Метод вспомогательного псевдокласса.

```
<!-- Добавляем класс clearfix к контейнеру -->
```

```
<div class="container clearfix">
 <div class="box" style="float: left;">Блок 1</div>
 <div class="box" style="float: left;">Блок 2</div>
</div>
```

```
.clearfix::after {
 content: ""; /* Создаем пустое содержимое */
 display: table; /* Или block, table надежнее для отступов */
 clear: both; /* Очищаем поток */
}
```

Псевдоэлемент ::after виртуально вставляется в конец контейнера и работает как очищающий блок из первого метода, но без лишнего HTML-тега.

# Clearfix-хаки. Метод свойства Flow-Root.

- **Как это работает:** Существует специальное значение свойства отображения, которое явно предписывает браузеру создать корневой контекст форматирования потока. Это заставляет контейнер автоматически включать в себя высоту всех дочерних элементов, даже если они изъяты из потока через обтекание.
- **Недостатки:** Основное ограничение — отсутствие поддержки в устаревших браузерах (например, Internet Explorer). Для современных проектов это не является проблемой, но при необходимости поддержки legacy-систем этот метод не подойдет.

# Clearfix-хаки. Метод свойства Flow-Root.

```
<div class="container">
 <div class="box" style="float: left;">Блок 1</div>
 <div class="box" style="float: left;">Блок 2</div>
</div>
```

```
.container {
 display: flow-root; /* Нативное создание контекста форматирования */
}
```

Браузер автоматически понимает, что этот контейнер должен содержать в себе все дочерние элементы, игнорируя их обтекание при расчете высоты.

# Clearfix-хаки. Метод отказа от обтекания.

- **Как это работает**: Вместо свойства **float** для создания колонок и сеток используются современные модули верстки. В этих моделях элементы по умолчанию не выпадают из потока, поэтому контейнер всегда корректно рассчитывает свою высоту на основе содержимого без каких-либо дополнительных инструкций.
- **Недостатки**: Требуется переосмысления подхода к верстке. Свойство **float** следует оставлять только для его прямой задачи — обтекания текстом изображений внутри статьи, а не для построения структуры страницы.

# Clearfix-хаки. Метод отказа от обтекания.

```
<div class="container">
 <div class="box">Блок 1</div>
 <div class="box">Блок 2</div>
</div>
```

```
.container {
 display: flex; /* Включаем гибкую верстку */
}
```

```
.box {
 /* float: left; – убираем эту строку, она больше не нужна */
 flex: 1; /* Растягиваем блоки равномерно */
}
```

Во Flexbox элементы не выпадают из потока, поэтому контейнер всегда имеет правильную высоту автоматически.